

Since you're learning ML and Linear Regression from scratch, here's a very simple implementation that demonstrates:

- Why we use a `class`
- Why we use `__init__()`
- Why we use `self`
- Why we don't use normal local variables
- Why we don't use global variables
- How Linear Regression actually calculates **m** (slope) and **b** (intercept)

Complete Code

</> Python

Run

```
import numpy as np

class MeraLR:

    # Constructor
    def __init__(self):

        # These belong to the object
        self.m = None
        self.b = None

        print("Model Created")
        print("m =", self.m)
        print("b =", self.b)
        print()

    # Training Function
    def fit(self, X_train, y_train):

        num = 0
        den = 0

        x_mean = X_train.mean()
        y_mean = y_train.mean()

        # Formula for slope
        for i in range(len(X_train)):

            num = num + ((X_train[i] - x_mean) *
                          (y_train[i] - y_mean))

            den = den + ((X_train[i] - x_mean) ** 2)

        # Calculate slope
        self.m = num / den
```

```

# Calculate intercept
self.b = y_mean - (self.m * x_mean)

print("Training Completed")
print("m =", self.m)
print("b =", self.b)
print()

# Prediction Function
def predict(self, X_test):

    y_pred = []

    for x in X_test:
        y = self.m * x + self.b
        y_pred.append(y)

    return np.array(y_pred)

```

Using the Model

Python



Run

```

X_train = np.array([1,2,3,4,5])
y_train = np.array([3,5,7,9,11])

model = MeraLR()

model.fit(X_train,y_train)

X_test = np.array([6,7,8])

predictions = model.predict(X_test)

print("Predictions:")
print(predictions)

```

Output

Model Created

```

m = None
b = None

```

Training Completed

```

m = 2.0
b = 1.0

```

Predictions:

```
[13. 15. 17.]
```

But imagine comparing models

Suppose later you want to compare two Linear Regression models.

Model A:

```
</> Python
model1.fit(train_data_1)
```

Model B:

```
</> Python
model2.fit(train_data_2)
```

Now each has different values:

```
</> Python
model1.m = 2
model1.b = 1

model2.m = 5
model2.b = 3
```

You can compare their predictions.

one object means one model since you are saying `model1=meraLR()` then it means you have created an object and that object using `self` keyword in `init` constructor will initialize the coefficients with initial values and in `fit` again `self` is used cause we want to give data to that objects coeff so it means we are talking about that single model and we are using `init` and `self` so that in case we want to compare different models then simply using object creation and initialization using `self` we can make it happen.

I hope you have understood it well

thank you

peace

Abhinav Srivastava

What happens when object is created?

When you write:

```
</> Python
model = MeraLR()
```

Python internally does:

```
</> Python
model = MeraLR()

MeraLR.__init__(model)
```

Therefore inside constructor:

```
</> Python
self == model
```

So:

```
</> Python
self.m = None
```

actually means:

```
</> Python
model.m = None
```